VMix record ingest — near-real-time, Nextcloud-aware

Goal: same as `docs/atem-iso-ingest.md` — get each VMix PC's growing recording onto the mothership's Nextcloud as close to real time as possible, without corrupting anything and without falling permanently behind. Covers all 4 VMix PCs: VMix Node 1 (`192.168.20.21` / `.22` , off Theatre 1) and VMix Node 2 (`192.168.21.21` / `.22` , off Theatre 4) — see `docs/ip-address-map.md` .

Why this is actually simpler than the ATEM case

The ATEM Mini Extreme ISO only exposes its recording drive over FTP — a file-transfer protocol, not a filesystem-access protocol, so a mount-based approach there needs `rclone mount` as a FUSE bridge (that's the ATEM doc's *alternative* path; its default pulls via FTP `REST` resume with no mount at all). **VMix PCs run Windows, which natively serves SMB** — a real network filesystem protocol, designed to be mounted. There's no bridging trick needed here; SMB *is* the native way two machines share a filesystem on a LAN. That makes this the more natural fit for a mount-based approach, not a stretch of the ATEM pattern onto a different protocol.

Architecture

Same shape as the ATEM ingest, reusing the identical tools (`config/vmix-record-ingest/` mirrors `config/atem-iso-ingest/rclone-mount-rsync/`), as its own Unraid container (`192.168.1.18`):

```
VMix PC (192.168.2X.2X, SMB share) --[Tailscale, via that VMix node's own A-1300]--> Unraid
                                                                  |
                                                        rclone mount (SMB backend)
                                                                  |
                                                                  v
                                                    rsync --append into Nextcloud's
                                                    External Storage (Local) folder
                                                                  |
                                                       targeted `occ files:scan`
                                                                  |
                                                                  v
                                                         visible in Nextcloud, growing
```

`rsync --append` (not rsync's general checksum-diff algorithm) is what makes this incremental — it seeks straight to the destination's current size and transfers only the new tail, rather than reading/hashing the whole file. Same reasoning as the ATEM ingest's `rclone-mount-rsync` alternative; see that doc for the full explanation of why this differs from rsync's usual delta-transfer mode.

Second write destination for live editing. Same as the ATEM ingest: the pull also writes to the edit suite's dedicated NAS (`192.168.22.x`) alongside Nextcloud's External Storage, one read off the VMix PC's SMB share, two writes. See `docs/live-editing.md` for the editing subsystem this feeds — not yet implemented in `config/vmix-record-ingest/mount-and-sync.sh`, tracked in `docs/open-questions.md`.

A native Linux CIFS mount (`mount -t cifs`) is a simpler alternative to `rclone mount` for SMB specifically — no FUSE bridge needed at all, since the kernel has first-class SMB client support. The tooling here uses rclone anyway, to reuse the exact same config/script pattern already built for the ATEM ingest rather than introduce a third mechanism. Worth reconsidering if operational simplicity outweighs consistency.

Why bandwidth isn't a shared concern with the ATEM ingest

Each VMix node has **its own dedicated GL-iNet A-1300 uplink and its own Tailscale connection** — it is not part of the theatre subnet it physically sits near (see `docs/topology.md`). So VMix Node 1's record-ingest traffic rides over Node 1's own link, entirely separate from Theatre 1's ATEM ISO ingest and SRT traffic. There's no contention between the two ingest pipelines sharing a link — each just needs to fit inside its own node's ~170 Mbps ceiling.

Open items

- **Confirm what VMix is actually recording** — program mix only, or per-input ISO the same way the ATEMs do — and at what resolution/codec/bitrate. None of this is assumed here; it directly determines the real bandwidth load on each VMix node's uplink.
- **Set up SMB sharing on the real PCs** — confirm the actual recording folder path and share name, and use a dedicated, minimal-privilege (read-only) local account for the ingest process rather than an admin/shared login.
- **Same partial-file-safety question as the ATEM ingest, but easier to answer**: VMix recording formats are more commonly designed for editability during capture than consumer camera formats, but this should still be empirically checked (open a currently-recording file's ingested copy in a player) rather than assumed.
- **Confirm the version-retention setting on Nextcloud** covers these folders too (see the same caveat in `docs/atem-iso-ingest.md`) — same risk of accumulating snapshots on a repeatedly-changing external-storage file.